

Universidad Nacional de Colombia
Facultad de Ingeniería – Departamento de Ingeniería de Sistemas
e Industrial
Computación Paralela y Distribuida
Tarea No. 03: Ejercicios – Paralelismo a Nivel de Tarea

Jonathan López

1. ¿Qué es un conjunto parcialmente ordenado (POSET)? Muestre cuáles serían las propiedades de un POSET con el conjunto de los divisores de un número entero.

1.1. ¿Qué es un POSET?

Un POSET es una estructura definida por $(S, \leq)$, donde S es un conjunto con al menos un elemento y $\leq$ es una relación binaria que cumple las siguientes propiedades:

- Reflexiva: $\forall a \in S, a \leq a$.
- Antisimétrica: $\forall a, b \in S, (a \leq b \wedge b \leq a) \implies a = b$.
- Transitiva: $\forall a, b, c \in S, (a \leq b \wedge b \leq c) \implies a \leq c$.

Cabe aclarar que, a diferencia de un conjunto totalmente ordenado, en un POSET pueden existir elementos $a, b \in S$ tales que no se cumple ni $a \leq b$ ni $b \leq a$. Se dice que estos elementos son incomparables y se denotan $a \parallel b$.

1.2. POSET con el conjunto de divisores de un número entero

Para $n \in \mathbb{Z}^+$, se define el POSET $D(n) = \{d \in \mathbb{Z}^+ : d \mid n \text{ (} d \text{ divide a } n)\}$, donde:

$$a \mid b \iff \exists k \in \mathbb{Z}^+ \text{ tal que } b = k \cdot a$$

Las propiedades de este POSET son:

- Reflexiva: $\forall a \in D(n), a \mid a$.
- Antisimétrica: $\forall a, b \in D(n), (a \mid b \wedge b \mid a) \implies a = b$.
- Transitiva: $\forall a, b, c \in D(n), (a \mid b \wedge b \mid c) \implies a \mid c$.

1.3. Ejemplo de POSET con el conjunto de divisores de 24: $D(24)$

Considerando $n = 24$, el conjunto de todos sus divisores positivos es:

$$D(24) = \{1, 2, 3, 4, 6, 8, 12, 24\}$$

Las relaciones de cobertura ($a < b$, dependencias directas en el orden) corresponden a:

$$\{1 \mid 2, 1 \mid 3, 2 \mid 4, 2 \mid 6, 3 \mid 6, 4 \mid 8, 4 \mid 12, 6 \mid 12, 8 \mid 24, 12 \mid 24\}$$

Los pares de **elementos incomparables** ($a \parallel b$, es decir, $a \nmid b$ y $b \nmid a$) corresponden a:

$$\{2 \parallel 3, 3 \parallel 4, 3 \parallel 8, 4 \parallel 6, 6 \parallel 8, 8 \parallel 12\}$$

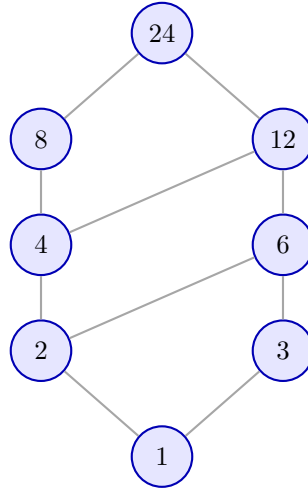


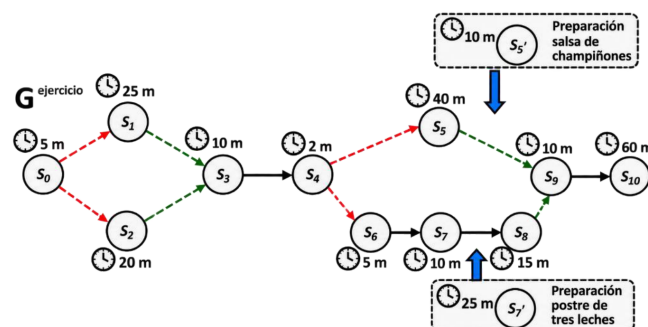
Figura 1: Diagrama de Hasse del POSET $(D(24), |)$.

En el diagrama de $D(24)$ podemos observar una correspondencia directa con un grafo computacional de tareas (DAG):

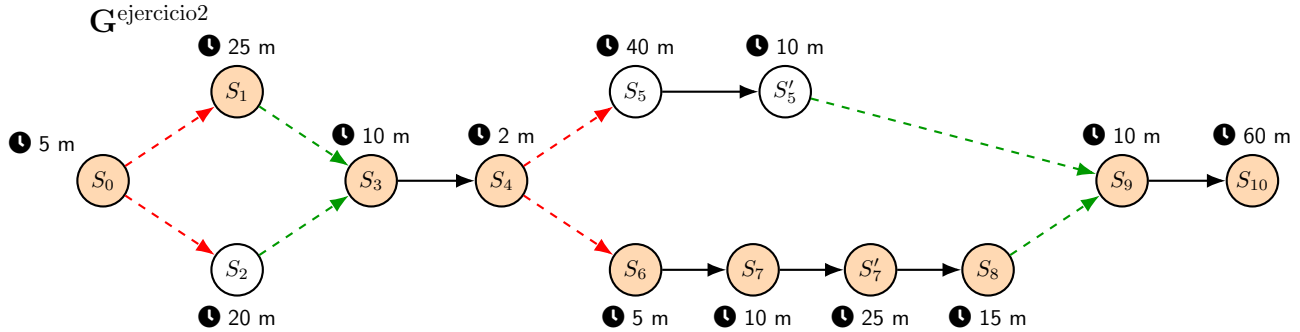
- **Relaciones de orden (cadenas):** Modelan dependencias de precedencia entre tareas secuenciales (por ejemplo, para procesar la tarea 4 se debe haber completado previamente la tarea 2, y antes de esta, la tarea 1).
- **Elementos incomparables (anticadenas):** Modelan tareas que no tienen dependencia mutua (como 4 y 6, o 8 y 12), por lo cual pueden ejecutarse de manera concurrente en procesadores independientes.

Por lo tanto, la estructura de un POSET proporciona el fundamento formal para modelar dependencias, sincronización y concurrencia en la ejecución de tareas en sistemas paralelos.

Responda las preguntas 2 y 3 con base en la siguiente figura:



2. Para el grafo computacional $G^{\text{ejercicio}}$ adicionarle las dos actividades indicadas en la ubicación que señalan las flechas azules en el diagrama. Después, calcule el $\text{WORK}(G^{\text{ejercicio}})$, el $\text{SPAN}(G^{\text{ejercicio}})$ y el paralelismo ideal correspondiente.



El trabajo es la suma de todos los tiempos que tomaría ejecutar todas las tareas en un único procesador; por lo tanto, el WORK es:

$$\text{WORK}(G^{\text{ejercicio}}) = \sum_i T(S_i) = 5 + 25 + 20 + 10 + 2 + 40 + 10 + 5 + 10 + 25 + 15 + 10 + 60 = 237 \text{ m}$$

El span es el camino secuencial más largo en el grafo (en este caso, el camino resaltado en naranja: $S_0 \rightarrow S_1 \rightarrow S_3 \rightarrow S_4 \rightarrow S_6 \rightarrow S_7 \rightarrow S'_7 \rightarrow S_8 \rightarrow S_9 \rightarrow S_{10}$); por lo tanto, el SPAN es:

$$\text{SPAN}(G^{\text{ejercicio}}) = 5 + 25 + 10 + 2 + 5 + 10 + 25 + 15 + 10 + 60 = 167 \text{ m}$$

Finalmente, el paralelismo ideal se define como WORK/SPAN ; por lo tanto:

$$\text{Paralelismo ideal} = \frac{\text{WORK}(G^{\text{ejercicio}})}{\text{SPAN}(G^{\text{ejercicio}})} = \frac{237}{167} \approx 1,42$$

3. Para el grafo computacional $G^{\text{ejercicio}}$, escribir el pseudocódigo de inicio y terminación de tareas utilizando las “palabras” `async` y `finish`.

En el modelo computacional estructurado con las primitivas `async` y `finish`:

- `async { Stmt }`: Crea una tarea hija asíncrona que se ejecuta concurrentemente. El hilo invocador (**hilo padre**) continúa inmediatamente con la siguiente instrucción secuencial.
- `finish { Stmt }`: Bloque de sincronización (*join*). El hilo padre ejecuta las instrucciones dentro del bloque y suspende su salida hasta que todas las tareas asíncronas creadas dentro del ámbito de dicho `finish` hayan terminado.

Para garantizar que el **hilo padre permanezca siempre ejecutando las actividades del camino crítico (SPAN)**:

$$S_0 \rightarrow S_1 \rightarrow S_3 \rightarrow S_4 \rightarrow S_6 \rightarrow S_7 \rightarrow S'_7 \rightarrow S_8 \rightarrow S_9 \rightarrow S_{10}$$

las tareas que se bifurcan por fuera de este camino crítico son las que se delegan al bloque `async`:

1. En la primera bifurcación, la actividad fuera del SPAN es S_2 (20 m). Por ende, se despacha S_2 en un `async`, y el hilo padre continúa directamente con S_1 (25 m).
2. En la segunda bifurcación, la rama fuera del SPAN es la superior $S_5 \rightarrow S'_5$ ($40 + 10 = 50$ m frente a $5 + 10 + 25 + 15 = 55$ m de la rama inferior). Por ende, se despacha dicha rama en un `async`, mientras el hilo padre ejecuta secuencialmente $S_6 \rightarrow S_7 \rightarrow S'_7 \rightarrow S_8$.
3. En los nodos secuenciales ($S_0, S_3, S_4, S_9, S_{10}$), el hilo padre continúa su ejecución secuencial directa.

```

S0(); // Hilo padre

finish {
    async {
        S2(); // Tarea concurrente
    }
    S1();      // El hilo padre continúa directamente por el SPAN
} // join: espera que termine S2

S3(); // Hilo padre
S4(); // Hilo padre

finish {
    async {
        S5();
        S5_prime(); // Tareas concurrentes
    }
    // El hilo padre continúa ejecutando
    S6();
    S7();
    S7_prime();
    S8();
} // join: espera que termine la rama de S5 y S5'

S9(); // Hilo padre
S10(); // Hilo padre finaliza la ejecución

```

Nota: Es igualmente válido y equivalente expresar ambas ramas explícitamente mediante `async` dentro del bloque `finish` (por ejemplo, `finish { async { S1(); } async { S2(); } }`).

4. El siguiente pseudo-código, suma dos matrices triangulares inferiores (matrices cuadradas $n \times n$ en las cuales los elementos por encima de la diagonal, incluyendo la diagonal de $(0,0)$ a (n,n) , son cero). En el código, cada ejecución de la sentencia `A[i][j] = B[i][j] + C[i][j]`; representa una unidad de tiempo en el grafo computacional correspondiente.

```

finish {
    for (int i = 0; i < n; i++) {
        async {

```

```
        for (int j = 0; j < i; j++) {  
            A[i][j] = B[i][j] + C[i][j];  
        } // for-j  
    } // async  
} // for-i  
} // finish
```

En términos de n ¿Cuál es el WORK del código mostrado? Explique cómo obtuvo su respuesta.

Iniciamos analizando de adentro hacia afuera: el bucle interno `for (int j = 0; j < i; j++)` ejecuta exactamente i iteraciones, donde cada iteración equivale a una unidad de tiempo. A su vez, el bucle externo `for (int i = 0; i < n; i++)` realiza exactamente n iteraciones (desde $i = 0$ hasta $i = n - 1$). En cada una de estas iteraciones, el bucle interno representa i unidades de tiempo. Así, la primera iteración del bucle externo ($i = 0$) toma 0 unidades, la segunda ($i = 1$) 1 unidad, la tercera 2 unidades, y así sucesivamente hasta la última iteración ($i = n - 1$) que toma $n - 1$ unidades de tiempo. Por lo tanto, el WORK total es la suma de todas estas unidades de tiempo:

$$\text{WORK} = 0 + 1 + 2 + \cdots + (n - 1) = \sum_{i=0}^{n-1} i = \frac{(n - 1)n}{2} = \frac{n^2 - n}{2}$$

Así, el WORK del código mostrado es $\frac{n^2 - n}{2}$ unidades de tiempo.